SPM Module 1: Vectorization of the Partition Mapping Kernel

Gabriele Marsili

1 Design Choices

Given N keys (`uint64_t`) and P partitions (power of two), the task is to compute part_id$[i] = h(\text{keys}[i]) \in [0, P)$ for every key.

1.1 Mapping Function

I chose a Fibonacci multiply-shift hash operating on 32 bits:

$$h(k) = \big((k_{\text{lo}} \oplus k_{\text{hi}}) \cdot A_{32}\big) \gg (32 - \log_2 P)$$

where $A_{32} = \texttt{0x9E3779B9} = \lfloor 2^{32}/\varphi \rfloor$ [1]. This is an instance of the *hashing by multiplication* scheme described in [2] (§ 8.2), where the golden-ratio constant $A = (\sqrt{5} - 1)/2 \approx 0.618$ is suggested for its bit-spreading properties. The right-shift extraction follows the *multiply-add-shift* universal hash family of [2] (§ 8.3): $h_{a,b}(k) = (ak + b \bmod 2^w) \operatorname{div} 2^{w-\ell}$, here simplified with $b = 0$ and the Fibonacci constant as a.

The XOR of the two 32-bit halves preserves information from both halves before the multiplication. The Fibonacci constant spreads bits well across the output range, and measured distribution on 10^8 keys with $P = 256$ gives max(count)/expected = 1.005.

The main reason for working in 32 bits is SIMD compatibility: `_mm256_mullo_epi32` (`vpmulld`) is a single native AVX2 instruction that multiplies 8 uint32 values at once. A 64-bit multiply in AVX2 would need 3 `vpmuludq` instructions plus shifts and adds per element, which I verified to be slower than the scalar baseline in this bandwidth-limited regime.

1.2 Data Layout and P

Keys and output partition IDs are in separate contiguous arrays with unit stride—the layout most amenable to vectorization. Memory is 32-byte aligned via `posix_memalign` for AVX2 aligned loads. P is a power of two so that modulo reduces to a right-shift.

2 Auto-vectorization

The plain C++ kernel is compiled from a single source file into two binaries: one with `-fno-tree-vectorize` (baseline), one with `-O3 -march=native -mavx2 -ftree-vectorize` (autovec). Both use `-fopt-info-vec-optimized` and `-fopt-info-vec-missed` for diagnostics.

GCC 12.2 reports successful vectorization of the kernel loop. The relevant lines from the full report (file `vec_report_optimized.txt`, included in the deliverables):

```
src/plain.cpp:33:26: optimized: loop vectorized
  using 32 byte vectors
src/plain.cpp:33:26: optimized: loop vectorized
  using 16 byte vectors
include/common.hpp:77:27: optimized: loop vectorized
  using 32 byte vectors
include/common.hpp:264:19: optimized: loop vectorized
  using 32 byte vectors
```

GCC emits two versions of the kernel loop: one using 256-bit AVX2 registers (`ymm`, 32-byte vectors, 8 uint32 per instruction) and one using 128-bit SSE registers (`xmm`, 16-byte vectors, 4 uint32 per instruction). At runtime the 256-bit path is taken when AVX2 is available; the 128-bit version serves as a fallback on older hardware. The `common.hpp` lines correspond to the checksum and key generation loops, which GCC also vectorizes. The `-fopt-info-vec-missed` report was empty—GCC found no loop that it attempted but failed to vectorize.

The loop satisfies all conditions: known trip count, no loop-carried dependencies, no function calls in the body, `__restrict__` to exclude aliasing, and unit stride access.

3 AVX2 Intrinsics

The hand-written version processes 8 keys per iteration. Two `ymm` loads bring in 4+4 uint64 keys. Each key is XOR-folded by shifting right 32 bits and XORing with the original value, leaving the folded result in the low 32 bits of each 64-bit lane. The 8 folded values are then packed into a single `ymm` register: `permutevar8x32` with indices `[0,2,4,6,1,3,5,7]` extracts the low-32 values from each register, and `permute2x128` with selector `0x20` combines the lower 128 bits of both into one register of 8×uint32. A single `vpmulld` performs all 8 Fibonacci multiplications, a right-shift extracts partition IDs, and a 256-bit store writes the results. The remaining N mod 8 keys are handled by a scalar tail.

The AVX2 binary verifies correctness against an internal scalar reference (compiled with `-fno-tree-vectorize` via pragma) by comparing FNV-1a checksums before benchmarking; for $N \leq 32$ it also prints element-wise comparison.

4 Performance Analysis

All benchmarks were run on node09 (AMD EPYC 7301, Zen 1, DDR4-2666) with exclusive SLURM allocation (`-partition=gpu-excl`), 11 repetitions, median reported.

4.1 Results

Table 1 reports throughput and speedup across five input sizes ($P = 256$).

	Baseline	Autovec		AVX2	
N	Mkeys/s	Mkeys/s	S	Mkeys/s	S
10^6	929	1377	1.48×	1276	1.37×
10^7	915	1274	1.39×	1172	1.28×
5×10^7	908	1314	1.45×	1168	1.29×
10^8	918	1310	1.43×	1200	1.31×
2×10^8	908	1322	1.46×	1152	1.27×

Table 1: CPU throughput and speedup vs. baseline at $P = 256$.

At $N = 10^8$ the CUDA kernel alone reaches 67 320 Mkeys/s (808 GB/s, 81% of A30 HBM2 peak), but end-to-end including PCIe transfers is only 1 031 Mkeys/s (1.12×); see Section 5.

4.2 Why the Speedup is Limited

Each key needs 12 bytes of memory traffic (8 B read + 4 B write) and only ∼4 integer operations. The operational intensity is $I = 4/12 \approx 0.33$ ops/byte, placing the kernel in the bandwidth-bound region of the roofline. The single-core bandwidth on node09 was measured at ∼16.4 GB/s; the autovec version reaches 15.7 GB/s, about 96% of this ceiling (Figure 1). SIMD speeds up the compute part, but memory transfers—which dominate—stay the same; the 1.43× speedup comes from more efficient bandwidth utilization, not faster computation.

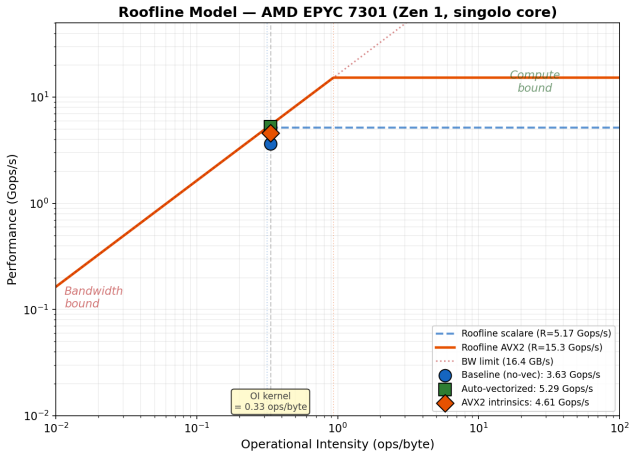

Figure 1: Roofline model. All variants sit on the bandwidth ramp ($I = 0.33$), well below compute ceilings.

4.3 Autovec vs. AVX2 Intrinsics

Somewhat counterintuitively, the compiler-vectorized version outperforms the hand-written intrinsics (15.7 vs 14.4 GB/s). Since the kernel is bottlenecked by memory, the marginal differences in instruction scheduling, loop unrolling, and register allocation that GCC applies during auto-vectorization translate into slightly better bandwidth utilization. The intrinsics code imposes a rigid instruction sequence that the compiler cannot easily rearrange.

4.4 Parameter Sweeps

N-sweep (Figure 3): speedup is stable across all input sizes, from 10^6 to 2×10^8 keys. This confirms that the kernel is bandwidth-bound regardless of N.

P-sweep (Figure 2a): throughput and speedup are independent of P (all curves flat across $P \in [2, 1024]$), confirming that changing P only modifies the shift constant—memory traffic and compute per key are unchanged.

Key-space sweep (Figure 2b): throughput is invariant to key-space size (duplicate rate), as expected for a branchless hash with constant cost per key.

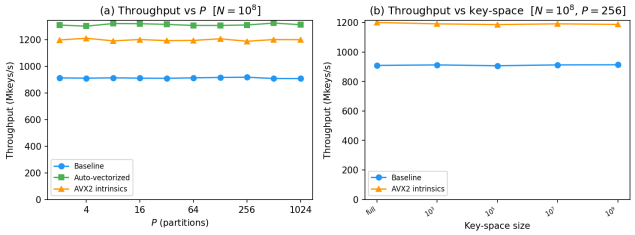

Figure 2: (a) Throughput vs. P at $N = 10^8$: flat across all partition counts. (b) Throughput vs. key-space size at $N = 10^8$, $P = 256$: invariant to duplicate rate.

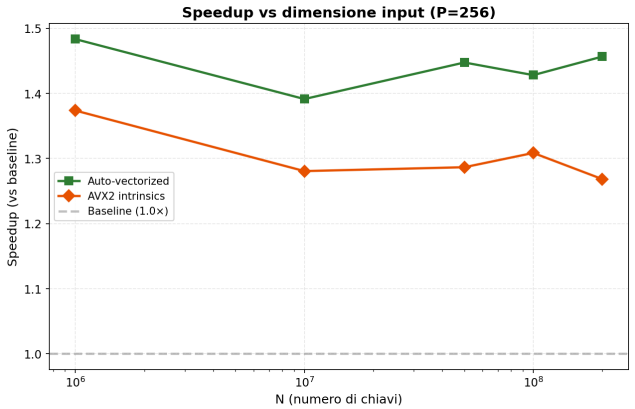

Figure 3: Speedup vs. input size N ($P = 256$). Both auto-vectorized and AVX2 intrinsics maintain stable speedup across all N, consistent with a bandwidth-bound kernel.

4.5 Measurement Stability

All 55 measurement points have a coefficient of variation below 5%. The autovec binary is the most stable (mean CoV 0.4%), while AVX2 intrinsics show slightly more variance (mean CoV 3.5%), likely because GCC's scheduling produces more deterministic memory access patterns than the hand-written sequence.

5 CUDA

The GPU kernel uses the same hash, one thread per key, 256 threads/block. Correctness is verified via FNV-1a checksum comparison against the CPU reference; for $N \leq 32$ element-wise comparison is also printed.

Phase	Time (ms)	BW (GB/s)	%
H→D transfer	63.2	12.7	65%
Kernel	1.5	808	2%
D→H transfer	32.3	12.4	33%
End-to-end	**97.0**	—	100%

Table 2: CUDA timing at $N = 10^8$, $P = 256$.

The kernel alone reaches 808 GB/s (81% of the A30's HBM2 peak of 993 GB/s). PCIe transfers account for 98% of end-to-end time, so offloading this isolated kernel gives no advantage over the CPU. GPU would pay off only with data already on device or with subsequent pipeline stages also on GPU.

6 Conclusions

This kernel is a textbook memory-bound workload. At $I = 0.33$ ops/byte, performance is dictated by DRAM bandwidth regardless of the SIMD strategy. The autovec version reaches 96% of the single-core bandwidth ceiling, leaving practically no room for single-thread improvement. The choice of a 32-bit hash (native `vpmulld`) over a 64-bit one was critical: with a 64-bit hash, AVX2 was actually slower than scalar due to the `vpmuludq` decomposition overhead in a bandwidth-limited regime.

References

[1] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998, § 6.4, pp. 513–558.

[2] P. Ferragina, *Pearls of Algorithm Engineering*. Cambridge University Press, 2023, Ch. 8: The Dictionary Problem, pp. 96–127.